

Project Documentation Report

A Project Report on

EduGenie

Gemini-Powered Learning Assistant

Submitted in partial fulfilment of the requirements of the **TN
Skills Portal Submission**

Submitted by

Jebin Jeron J

Inbarasan k

Sureas M

Hariharan R

Academic Year 2026

Acknowledgement

We express our sincere gratitude to the TN Skills initiative for the opportunity to design, build and document this project. Working on EduGenie allowed us to explore the practical application of Generative AI models to the domain of academic learning support, while strengthening our understanding of rapid web application development using Python, Flask and modern front-end technologies.

We would like to thank our faculty mentors and project guides for their valuable feedback and encouragement throughout the development of this project.

We also acknowledge the developers of Flask, the Google Generative AI SDK, and the Gemini model family, whose tools and platforms made the rapid development of this application possible.

Finally, we thank our peers and families for their continued support and encouragement.

Abstract

Academic learning increasingly demands quick, personalized access to explanations, summaries and practice material, yet students often struggle to find study support that adapts to their individual pace and phrasing of a question. EduGenie is proposed as an AI-powered learning assistant that addresses this gap by using Google's Gemini Generative AI model to provide simple explanations, topic summaries, study notes, examples and practice quizzes for any topic or question a student enters.

The application is designed as a full-stack web application using Python and Flask for the backend, HTML, CSS and JavaScript for the front-end, and the Google Generative AI SDK for communicating with the Gemini API. Given a topic or question entered by the student, EduGenie constructs a structured educational prompt and forwards it to the Gemini model, which returns a student-friendly response tailored to the selected learning mode — explanation, summary, example generation, or quiz creation.

A key design goal of EduGenie is to reduce the time and effort required for students to prepare study material by consolidating multiple learning activities — understanding a concept, summarizing it, seeing worked examples, and testing recall through quizzes — into a single, conversational web interface.

This report documents the design, architecture, technology stack, implementation, and testing of the EduGenie application. It also discusses the feasibility of the system, its advantages and limitations, and potential directions for future enhancement, such as multi-language learning support, a student progress dashboard, and PDF-based document learning.

The project demonstrates how a modern Generative AI model can be combined with a lightweight, easy-to-deploy Flask web application to build an accessible, student-friendly tool that lowers the barrier to effective self-directed learning.

Table of Contents

Chapter 1: Introduction Chapter 2:

Literature Survey Chapter 3:

System Analysis

Chapter 4: System Architecture and Design

Chapter 5: Technology Stack

Chapter 6: Implementation and Setup

Chapter 7: Feature Description Chapter

8: Testing

Chapter 9: Results, Discussion, Conclusion and Future Enhancements References

List of Figures and Tables

Figure 4.1: High-Level System Architecture of EduGenie Table

3.1: Software Requirements

Table 3.2: Hardware Requirements Table

8.1: Test Cases and Results

Chapter 1: Introduction

1.1 Overview

Learning a new academic concept traditionally requires access to a teacher, a well-organized textbook, or curated study notes, along with the time to read, summarize and self-test on that material. However, many students — particularly those preparing independently for exams, revising between classes, or exploring a topic outside of formal instruction — do not have consistent access to a tutor who can explain a concept in simple language at the moment they need it.

EduGenie addresses this challenge by using Google's Gemini Generative AI model to provide on-demand, student-friendly learning assistance. The application handles the cognitive heavy-lifting of study preparation — explaining concepts in simple language, generating concise notes and summaries, producing worked examples, and creating multiple-choice practice questions — allowing any student to receive tailored learning support within seconds, through a simple and intuitive web-based interface.

1.2 Problem Statement

Students frequently encounter topics they do not fully understand but lack immediate access to a teacher, tutor, or well-structured study material to clarify the concept. Manually preparing study notes, generating practice questions, and finding suitable examples for a topic is a time-consuming process that many students, especially those balancing multiple subjects or deadlines, are unable to complete effectively on their own.

Additionally, most existing AI writing and chat tools respond with generic, unstructured text rather than content organized specifically for learning — such as a clear definition, a step-by-step explanation, a worked example, and follow-up practice questions — leaving students to manually reorganize the AI's response into something they can actually study from.

1.3 Objectives

To design and implement a Flask-based web application that allows students to receive AI-generated explanations, summaries, examples and quizzes for any topic or question they enter.

To integrate Google's Gemini model, via the Google Generative AI SDK, to perform the underlying natural language understanding and content generation.

To structure the AI-generated output into distinct, well-defined learning modes: topic explanation, note/summary generation, worked examples, and quiz/practice question generation.

To implement structured prompt engineering that ensures explanations are simple, age-appropriate, and organized into clear, step-by-step content suitable for student revision.

To provide a simple, responsive web interface requiring no technical knowledge from the end user.

To securely manage the Gemini API credentials using environment variables via python-dotenv.

1.4 Scope of the Project

The scope of this project covers the design and development of the EduGenie application as a Flask-based web tool that generates textual learning content — explanations, summaries, examples, and quiz questions — using the Gemini model. The current implementation focuses on text-based learning assistance and does not include voice input, automatic document/PDF ingestion, or persistent student accounts.

Multi-user account management, learning-history persistence in a database, and a personalized progress-tracking dashboard are outside the scope of the current version and are discussed as potential future enhancements in the concluding chapter of this report.

1.5 Organization of the Report

This report is organized into nine chapters. Chapter 1 introduces the project. Chapter 2 surveys related work in AI-assisted learning and educational chatbot tools. Chapter 3 discusses system analysis, including feasibility. Chapter 4 presents the system architecture and design, including a system diagram. Chapter 5 describes the technology stack. Chapter 6 details the implementation and setup procedure. Chapter 7 describes the core features of the application. Chapter 8 covers testing methodology. Chapter 9 presents results, discussion, conclusion and future enhancements, followed by references.

Chapter 2: Literature Survey

Prior to designing EduGenie, a review of existing AI-assisted learning and educational chatbot tools was conducted to identify design opportunities and gaps that this project could address.

2.1 General-Purpose AI Chat Assistants

General-purpose AI chat assistants built on large language models can answer academic questions when prompted directly. However, these tools typically return a single, unstructured block of conversational text rather than content organized around a specific learning mode — such as a definition-first explanation, a condensed summary, or a set of practice questions — requiring the student to manually extract and reorganize the useful parts of the response.

2.2 E-Learning Platforms and Adaptive Tutoring Systems

Academic and industry research into intelligent tutoring systems has explored combining language models with adaptive learning paths, progress tracking, and personalized content recommendations. While promising, many such systems are large, institutionally deployed platforms requiring significant infrastructure and are not readily accessible as lightweight, self-hostable tools for individual student use or small academic projects.

2.3 Low-Code / No-Code Web Frameworks for AI Applications

Lightweight web frameworks such as Flask have become popular for rapidly prototyping AI-powered applications, as they allow a small development team to build a functional front-end and backend around a machine learning or LLM API with minimal boilerplate code. This trend directly informed the choice of Flask, combined with a simple HTML, CSS and JavaScript front-end, as the interface framework for EduGenie, prioritizing rapid development and ease of deployment over a fully custom front-end framework.

2.4 Identified Gaps

The survey identified the following gaps: (1) a lack of lightweight, easily deployable tools that generate learning content structured specifically around distinct study activities — explanation, summary, examples, and quiz — rather than a single block of generic prose; (2) limited attention to simple-language, student-friendly phrasing in general-purpose AI chat tools; and (3) minimal focus on generating ready-to-use practice questions alongside explanatory content, which would bridge the gap between understanding a concept and testing recall of it.

EduGenie addresses these gaps by using a structured, mode-based generation approach that explicitly separates learning content into explanation, summary, example, and quiz components, while keeping the language simple and student-appropriate, all within a minimal, easy-to-run Flask application.

Chapter 3: System Analysis

3.1 Proposed System

The proposed system, EduGenie, is a Flask-based web application that presents a simple, student-friendly interface where a learner enters a topic or question and selects a desired learning mode. This input is sent to the Google Gemini model via the Google Generative AI SDK, using a carefully constructed prompt that instructs the model to respond in a simple, structured, and student-appropriate format based on the selected mode — explanation, summary/notes, examples, or quiz.

The generated content is displayed within the web interface in a readable format, and the student can continue the interaction by asking follow-up questions or switching to a different learning mode for the same topic, supporting an interactive and iterative study session.

3.2 Feasibility Study

3.2.1 Technical Feasibility

The technology stack used in EduGenie — Python, Flask, HTML/CSS/JavaScript, and the Google Generative AI SDK — is well-documented, actively maintained, and widely used in the rapid AI-application development community. Flask's minimal, unopinionated design allows a fully functional web interface and backend API to be built with a small amount of code, making the project technically feasible for a small development team.

3.2.2 Economic Feasibility

EduGenie relies on open-source software (Python, Flask, python-dotenv) and the Google Gemini API, which offers a free usage tier suitable for development, demonstration, and light personal use. There are no licensing costs associated with the technologies used, and hosting a Flask application is inexpensive, with several free or low-cost hosting options available for small-scale deployment, making the project highly economical.

3.2.3 Operational Feasibility

The application's simple, form-based interface requires no training for end users — a student simply enters a topic or question, selects a learning mode, and receives generated content. The minimal operational footprint (a small Flask backend, a set of HTML templates, and a small set of dependencies) also makes the application easy to maintain, update, and redeploy.

3.3 Software and Hardware Requirements

Table 3.1: Software Requirements

Requirement	Detail
Operating System	Windows 10/11, macOS, or Linux
Runtime	Python 3.9 or later

Package Manager	pip
Backend Framework	Flask
Front-End	HTML5, CSS3, JavaScript
SDK	Google Generative AI SDK (google-generativeai)
Utility Library	python-dotenv
Development Tools	Visual Studio Code or any Python IDE
Version Control	Git and GitHub

Table 3.2: Hardware Requirements

Requirement	Detail
Processor	Intel Core i3 / AMD equivalent or higher (minimum)
RAM	4 GB minimum, 8 GB recommended
Storage	Less than 500 MB for dependencies and application files
Internet Connectivity	Required for Gemini API calls

3.4 Development Methodology

The project followed a Rapid Application Development (RAD) approach, well suited to a small, student-built application centered around a rapidly evolving AI model. Given the lightweight nature of the Flask framework, the development cycle emphasized quick prototyping of the prompt design and UI layout, followed by iterative refinement based on the quality, clarity and simplicity of the AI-generated learning content.

Each iteration involved refining the prompts sent to the Gemini model to improve the simplicity, structure, and educational quality of the generated explanations, summaries, examples and quizzes, testing the change interactively within the web interface, and incorporating improvements based on observed output quality.

Chapter 4: System Architecture and Design

4.1 Architecture Overview

EduGenie follows a simple, client-server architecture centered around a Flask backend application that interfaces with the Google Gemini model through the Google Generative AI SDK, and a lightweight HTML/CSS/JavaScript front-end that communicates with the backend over HTTP. Despite its architectural simplicity, the internal content-generation logic is organized into clearly defined conceptual stages to ensure structured, student-appropriate output. Figure 4.1 illustrates the high-level architecture of the system.



Figure 4.1: High-Level System Architecture of EduGenie

4.2 Component-wise Architecture Description

4.2.1 Presentation Layer (HTML / CSS / JavaScript)

The presentation layer is implemented using templates/index.html, styled with static/css/style.css

and made interactive using `static/js/script.js`. It renders the topic/question input field, the learning-mode selector, displays the generated content, and provides quiz interaction controls. JavaScript handles asynchronous requests to the backend so the page can update without a full reload.

4.2.2 Configuration Layer (.env)

The `.env` file, loaded via `python-dotenv`, securely stores the `GEMINI_API_KEY` required to authenticate with the Google Gemini API. Keeping this credential outside of the source code prevents accidental exposure of the API key in version control and allows different environments (development, demonstration) to use different keys without code changes.

4.2.3 Backend / API Layer (Flask - app.py)

The backend, implemented in `app.py`, exposes routes that receive the student's topic or question from the front-end, validate the input, and route the request to the appropriate learning-mode handler. It is responsible for orchestrating the call to the Gemini model and returning the generated content back to the front-end as a JSON or rendered HTML response.

4.2.4 Prompt Construction Engine

The Prompt Construction Engine builds a structured prompt for each learning mode based on the student's topic. For an explanation request, the prompt instructs the model to give a clear definition, a step-by-step explanation, a simple example, and key points. For a quiz request, the prompt instructs the model to produce a fixed number of multiple-choice questions, each with four options, the correct answer, and a short explanation.

4.2.5 AI Integration Layer (Google Generative AI SDK)

The AI integration layer uses the Google Generative AI SDK to send the constructed prompt to the Gemini model and to receive the generated response. This layer is responsible for handling the API request lifecycle, including authentication using the loaded `GEMINI_API_KEY` and basic error handling for failed or malformed requests.

4.2.6 Response Processing Layer

The Response Processing Layer takes the raw text response from the Gemini model and formats it for display — for example, separating a quiz response into individual question blocks with labeled options, or preserving the headings and bullet points of an explanation so it renders clearly in the browser.

4.3 Data Flow

The data flow within EduGenie proceeds as follows: (1) The student enters a topic or question into the web input form and selects a learning mode. (2) On submission, JavaScript sends the input to the Flask backend via an HTTP request. (3) `app.py` constructs a structured prompt appropriate to the selected learning mode. (4) This prompt is sent to the Gemini model via the Google Generative AI SDK, authenticated using the `GEMINI_API_KEY` loaded from the `.env` file. (5) The model's response is received and processed by the Response Processing Layer. (6) The formatted content is returned to the front-end and displayed to the student within the same page, without requiring a

full reload.

Because EduGenie does not persist data to an external database in its current version, each session is stateless beyond the current browser session — every new topic or question submitted results in an independent generation cycle, which simplifies the overall architecture considerably compared to a multi-user, persistent web application.

4.4 Use Case Description

The primary actor in the EduGenie system is the Student. The key use cases supported include: Enter Topic or Question, Select Learning Mode, Generate Explanation, Generate Summary/Notes, Generate Examples, Generate Quiz, and View Generated Content. Each use case is realized through direct interaction with the single-page web interface, which internally coordinates the call to the Gemini API and the subsequent processing and display of the result.

For example, in the 'Generate Quiz' use case, the student submits a topic and selects the quiz mode; the application constructs and sends the corresponding prompt to Gemini requesting five multiple-choice questions with options, correct answers, and explanations; the response is parsed into individual question blocks; and the complete quiz is displayed to the student within the same interface, without requiring a page reload.

Chapter 5: Technology Stack

EduGenie is intentionally built using a minimal but effective technology stack, prioritizing rapid development, ease of deployment, and straightforward maintenance. This chapter describes each of the core technologies used.

5.1 Frontend / Interface

HTML5: Used to structure the single-page learning interface, including the topic input form, mode selector, and content display areas.

CSS3: Used to style the interface, providing a clean, student-friendly, responsive design.

JavaScript: Used to handle user interactions, send asynchronous requests to the Flask backend, and dynamically update the page with generated content without a full reload.

5.2 Core Application

Python: The core programming language used to implement the entire backend application logic, including prompt construction, API communication, and response processing.

Flask: A lightweight Python web framework used to implement the backend routes, handle incoming requests from the front-end, and render the HTML templates.

python-dotenv: Used to load environment variables, specifically the `GEMINI_API_KEY`, from a local `.env` file, keeping sensitive credentials out of the source code.

5.3 Artificial Intelligence Stack

Google Gemini API: The underlying Large Language Model used to generate learning content, including explanations, summaries, notes, examples, and quiz questions, based on a structured prompt derived from the student's input.

Google Generative AI SDK: The official Python SDK used to authenticate with and send requests to the Google Gemini model, and to receive and process the generated text response within the application.

5.4 Data Persistence

EduGenie does not use a persistent database in its current version. Generated learning content exists only within the active browser session and is displayed directly to the student, rather than being stored server-side.

5.5 Development Tools

requirements.txt: Used to declare and pin all Python package dependencies, ensuring that the application can be reliably installed in a new environment with a single pip command.

Git and GitHub: Used for version control and source code management throughout the development of the project.

Visual Studio Code: Used as the primary development environment for writing and debugging app.py, the HTML templates, and the front-end assets.

Chapter 6: Implementation and Setup

This chapter describes the practical setup and implementation procedure for running EduGenie, based on the project's structure of a Flask application (app.py), HTML templates, static assets, a requirements.txt dependency manifest, and a .env configuration file.

6.1 Environment Configuration

Before running the application, a .env file must be created in the root of the EduGenie project directory. This file must define the GEMINI_API_KEY variable, containing a valid API key obtained from Google's Generative AI platform. This key is loaded at runtime using python-dotenv and used to authenticate all requests made to the Gemini model, ensuring that the credential is never hard-coded into app.py or committed to version control.

6.2 Installation and Launch Steps

Step 1 — Clone or download the repository — The developer obtains the EduGenie project files, including app.py, requirements.txt, templates/index.html, static assets, and README.md, either by cloning the GitHub repository or downloading the project archive.

Step 2 — Create a virtual environment (recommended) — A dedicated Python virtual environment is created for the project using "python -m venv venv" to isolate EduGenie's dependencies from the system-wide Python installation.

Step 3 — Activate the virtual environment — The environment is activated (for example, using "venv\Scripts\activate" on Windows or "source venv/bin/activate" on macOS/Linux) so that subsequent package installations are scoped to the project.

Step 4 — Install dependencies — Running "pip install -r requirements.txt" installs all required Python packages, including Flask, the Google Generative AI SDK, and python-dotenv, as declared in the project's dependency manifest.

Step 5 — Configure the .env file — The developer creates a .env file in the project root and adds the line "GEMINI_API_KEY=<your_api_key_here>", replacing the placeholder with a valid Gemini API key.

Step 6 — Launch the application — Running "python app.py" starts the Flask development server, typically accessible at http://127.0.0.1:5000, and automatically serves the EduGenie interface to the browser.

Step 7 — Enter a topic or question — Once the application is running, the student enters a topic or question into the provided input field, selects a learning mode, and submits the form, triggering the application to call the Gemini API and display the generated content.

Step 8 — Continue the learning session — After content is generated, the student can enter a follow-up question, switch to a different learning mode for the same topic, or start a new topic, without needing to restart the application.

6.3 Application Internals

Internally, `app.py` is organized around a small number of core responsibilities: defining Flask routes that serve the HTML interface and accept topic/question submissions; constructing a detailed prompt string appropriate to the selected learning mode (explanation, summary, examples, or quiz); invoking the Google Generative AI SDK to send this prompt and receive the model's response; performing light processing of the returned text so it displays cleanly; and returning the result to the front-end for rendering.

This straightforward, small-footprint design keeps the application easy to understand, debug, and extend, which is well suited to the project's goal of providing a lightweight, accessible learning-support tool rather than a large-scale, multi-service system.

Chapter 7: Feature Description

This chapter describes the core features of EduGenie in detail, building on the architectural description provided in Chapter 4.

7.1 AI Learning Assistant

The central feature of EduGenie is its ability to act as an on-demand learning assistant that responds to any topic or question a student enters, adapting its response format based on the learning mode the student selects.

7.2 Question and Answer Support

Students can type a specific question about a concept, and EduGenie generates a direct, simple-language answer, helping to resolve doubts that might otherwise require waiting for a teacher or tutor to be available.

7.3 Topic Explanation

For a given topic, EduGenie generates a clear definition, a step-by-step explanation of the concept, a simple example, and a list of important points, following the structured educational prompt design described in Chapter 4.

7.4 Automatic Notes / Summary Generation

EduGenie can condense a topic into a concise summary or set of study notes, helping students quickly revisit key points before an exam without re-reading full explanations.

7.5 Quiz / Practice Question Generation

EduGenie generates multiple-choice practice questions for a given topic, each with four options, the correct answer clearly identified, and a short explanation of why that answer is correct, allowing students to self-test their understanding.

7.6 Personalized Learning Prompts

The prompt design allows EduGenie's responses to be tailored to a student-appropriate reading level, ensuring explanations remain accessible regardless of how technically the original question was phrased.

7.7 Google Gemini Integration

EduGenie integrates directly with the Google Gemini Generative AI model via the official SDK, providing high-quality natural language understanding and generation without requiring any custom model training.

7.8 Simple, Student-Friendly Interface

The entire application is accessible through a single, uncluttered web page, requiring no technical knowledge from the end user beyond typing a topic into a text box, selecting a mode, and clicking a button, making the tool accessible to school students, college students, and self-learners alike.

7.9 Real-Time AI Responses

Because the application calls the Gemini API directly and displays the result asynchronously via JavaScript, students receive generated learning content within seconds of submitting their topic or question, supporting a fluid, conversational study session.

Chapter 8: Testing

Testing for EduGenie focused on validating both the functional correctness of the Flask application and the qualitative structure, clarity, and simplicity of the AI-generated learning content, given the inherently non-deterministic nature of Large Language Model output.

8.1 Levels of Testing

8.1.1 Functional Testing

Functional testing verified that the web interface correctly accepts student input, that the application correctly constructs and sends prompts to the Gemini API for each learning mode, that valid responses are correctly received, and that the interface updates without requiring a page reload.

8.1.2 Output Structure Testing

Given that the AI-generated response is free-form text, testing was conducted to confirm that the response formatting logic in app.py reliably renders explanations, summaries, examples and quiz questions in a clean, readable format across a variety of different topics, and handles minor variations in the model's response formatting gracefully.

8.1.3 Content Simplicity Testing

Multiple test topics were submitted to qualitatively evaluate whether the generated explanations remained in simple, student-appropriate language, confirming the effectiveness of the prompt design described in Chapter 4.

8.1.4 Error Handling Testing

Testing was performed to confirm that the application handles error conditions gracefully, such as a missing or invalid GEMINI_API_KEY, network connectivity issues when contacting the Gemini API, and empty or extremely short user input, displaying an informative message to the user rather than an unhandled application crash.

8.2 Test Cases

Table 8.1: Test Cases and Results

Test Case ID	Description	Expected Result	Status
TC-01	Submit a valid topic in Explanation mode	Clear definition, step-by-step explanation and example generated and displayed	Pass
TC-02	Submit an empty topic field	Validation message shown; no API call made	Pass
TC-03	Run application with missing GEMINI_API_KEY	Clear configuration error displayed to user	Pass
TC-04	Submit topic with no internet	Graceful error message displayed	Pass

TC-05	connection	Five MCQs generated with options, correct answers and explanations	Pass
TC-06	Submit multiple different topics sequentially	Each generates an independent, correctly formatted response	Pass
TC-07	Review generated explanation for simplicity	Explanation uses simple, student-appropriate language	Pass
TC-08	Submit a very long or complex topic	Application handles the request gracefully without crashing	Pass

Chapter 9: Results, Discussion, Conclusion and Future Enhancements

9.1 Results and Discussion

EduGenie was successfully implemented and demonstrated to reliably transform short student-provided topics and questions into simple, structured learning content, including explanations, summaries, examples and quiz questions. The lightweight Flask architecture proved highly effective for this use case, enabling rapid development and an intuitive, distraction-free student experience.

The prompt-based approach to controlling response format, implemented purely through careful prompt engineering rather than any explicit content-management system, was found to be effective across a range of test topics, producing consistently simple and well-structured learning content appropriate for student use.

The separation of the application into distinct learning modes — explanation, summary, examples, and quiz — proved valuable both for the clarity of the interface and for the practical usability of the output, allowing students to quickly move between understanding a concept and testing their recall of it.

Overall, the results confirm that a lightweight, Flask-based architecture combined with the Google Gemini API is well suited to building accessible, student-friendly learning tools, achieving the objectives outlined in Chapter 1 with a minimal technology footprint.

9.2 Advantages

- Extremely simple deployment and setup, requiring only a Python environment, a small set of dependencies, and a Gemini API key.

- Multiple structured learning modes — explanation, summary, examples, and quiz — directly usable for study, rather than unstructured prose.

- Simple, student-appropriate language achieved without complex fine-tuning or content-filtering infrastructure.

- Ready-to-use quiz questions with answers and explanations, supporting active self-testing rather than passive reading.

- Accessible, no-technical-knowledge-required interface suitable for school students, college students and self-learners.

- Low operating cost, relying on the free tier of the Gemini API and free or low-cost Flask hosting options.

9.3 Limitations

- The application does not currently support document or PDF-based learning; students must type their topic or question manually.

As with any LLM-based system, occasional inconsistencies or lower-quality output can occur, particularly for very unusual or ambiguous topics.

The application does not persist generated learning content server-side; there is no account system or learning history.

The current version does not track student progress over time or adapt difficulty based on prior performance.

9.4 Conclusion

This project presented the design, architecture, and implementation of EduGenie, an AI-powered learning assistant that uses Google's Gemini model to help students understand, summarize, and revise academic topics. By combining a lightweight Flask backend with a simple HTML, CSS and JavaScript front-end and the Google Generative AI SDK, the project demonstrates how a minimal, easy-to-deploy web application can deliver a genuinely useful and accessible learning tool with minimal engineering overhead.

The application successfully fulfills its stated objectives: generating simple, structured explanations, summaries, examples and quizzes for any student-provided topic, while requiring no account setup or technical knowledge from the user, all through a simple and user-friendly web interface. The project illustrates the value of careful prompt engineering as a lightweight approach to guiding a general-purpose Large Language Model toward a specific, student-focused output format.

EduGenie serves as a practical demonstration of applying modern Generative AI to academic learning support, and provides a solid, extensible foundation for a more comprehensive AI-assisted learning platform.

9.5 Future Enhancements

PDF upload and document-based learning: Allowing students to upload lecture notes or textbook pages and receive explanations, summaries and quizzes generated directly from that material.

Voice-based questions and text-to-speech answers: Enabling students to ask questions verbally and listen to generated explanations.

Multi-language learning support: Allowing students to receive explanations in their preferred regional language.

Student progress dashboard: Introducing lightweight tracking of topics studied and quiz performance to help students identify weak areas.

Personalized learning paths: Adapting the difficulty and depth of explanations based on a student's demonstrated understanding over time.

Automatic question-paper and flashcard generation: Extending the quiz module to generate full practice question papers and flashcard sets for revision.

AI study planner: Helping students organize their revision schedule across multiple

topics and subjects.

Student login and profiles: Introducing accounts and persistent storage to allow students to save, revisit, and build on previous learning sessions.

Deployment as a hosted service: Packaging and deploying EduGenie on a managed cloud platform for public access beyond local development use.

References

- [1] Flask Official Documentation — <https://flask.palletsprojects.com>
- [2] Google Gemini API Documentation — <https://ai.google.dev>
- [3] Google Generative AI SDK Documentation — <https://ai.google.dev/gemini-api/docs/sdks>
- [4] python-dotenv Documentation — <https://pypi.org/project/python-dotenv/>
- [5] Python Official Documentation — <https://docs.python.org/3/>
- [6] EduGenie Project Repository — <https://github.com/priya12pri2007-cpu/EduGenie-Gemini-Learning-Assistant>